

AI_reference.pdf

Assignment 02 - Web Mario

This report documents the use of AI assistance during development of the Cocos Creator 2.4.8 Mario-style game project.

AI Tool(s) Used

ChatGPT (OpenAI, GPT-5.5 Thinking) was used for programming guidance, debugging, code generation, README drafting, and this AI usage report.

Important Note About Animations

The final submitted project does not include Cocos Animation clips for the player or enemy. Therefore, the README lists animations as not implemented. Although AI assistance was used for other gameplay, UI, physics, and sound-related code, animation should not be claimed as a completed scoring item.

Scope of Usage Summary

File	AI-assisted purpose	Approximate lines in current project
assets/scripts/GameManager.ts	Game state flow, UI panel switching, score/life/timer, physics enabling, restart/reset, BGM/effects helpers.	1-318
assets/scripts/PlayerController.ts	Keyboard movement, jump, damage, respawn, enemy stomp handling, mushroom power-up, out-of-bounds death, sound hooks.	1-413
assets/scripts/EnemyController.ts	Enemy patrol, boundary turn-around, stomp/death behavior, reset behavior, physical collision suppression with player.	1-192
assets/scripts/BlockController.ts	Question block hit-from-below detection, mushroom spawning, used-block state, reset behavior, sound hook.	1-169
assets/scripts/MushroomController.ts	Mushroom movement, player collection, enemy collision avoidance, cleanup/reset behavior.	1-99
assets/scripts/GoalFlagController.ts	Level clear detection using contact listener and backup bounding-box check.	1-63
assets/scripts/CameraController.ts	Camera/background/UI behavior to avoid black screen and keep UI visible.	1-73
assets/scripts/MenuButtonController.ts	Start, level select, restart, and back-to-menu button handlers.	1-25
assets/scenes/Level1.fire	Scene property adjustment, node placement fixes, reset-related serialized values, removal of placeholder nodes/icons.	Scene asset edited through Cocos and/or script patching

AI Usage Entries

1. Initial game process and basic rules implementation

Location: GameManager.ts, PlayerController.ts, EnemyController.ts, BlockController.ts, CameraController.ts, MushroomController.ts, GoalFlagController.ts, MenuButtonController.ts; major portions of these files.

Prompt:

"this is the template and source files, please do the complete game process part and basic rules part and return a .zip file"

AI response / generated assistance: ChatGPT generated a first implementation for the game process and basic rules, including game manager, player movement, enemy behavior, question block behavior, mushroom behavior, camera behavior, goal flag behavior, and menu button behavior. The first generated version was later revised because the original template did not include a complete Cocos scene.

Refinement and explanation: I recreated and connected the scene manually in Cocos Creator 2.4.8. I assigned sprites, colliders, rigid bodies, labels, button click handlers, panels, and prefabs in the editor. I also tested gameplay repeatedly and asked for bug fixes when behavior did not match the project requirements.

2. Cocos Creator 2.4.8 conversion

Location: All TypeScript scripts, especially imports, decorators, physics classes, keyboard input, and audio usage.

Prompt:

"convert the scripts into cocos 2.4.8 format"

AI response / generated assistance: ChatGPT converted the earlier Cocos 3.x style scripts into Cocos Creator 2.4.8 style. The code changed from `import { _decorator, Component } from cc` style to const { ccclass, property } = cc._decorator` and extends cc.Component`. It also changed 2D physics classes and input APIs to the Cocos 2.4 equivalents.`

Refinement and explanation: I opened the project in Cocos Creator 2.4.8 and fixed script/component attachment problems manually. The controllers were attached directly to the nodes they control instead of child nodes.

3. Scene/node structure guidance

Location: Cocos scene setup, `Level1.fire` , node hierarchy, button click events, prefab setup.`

Prompt:

"please list the node structure and their node types" and "how to generate mushroom prefab"

AI response / generated assistance: ChatGPT provided a recommended scene hierarchy containing Canvas, Main Camera, GameManager, World, Player, Ground, Blocks, Enemies, Items, GoalFlag, BGM, and UI panels. It also explained how to create a Mushroom prefab and assign it to the question block script.

Refinement and explanation: I manually created the scene nodes in Cocos Creator, assigned SpriteFrames, configured RigidBody and PhysicsBoxCollider components, created UI labels/buttons, and set panel active states.

4. Physics and movement debugging

Location: GameManager.ts lines 66-77, PlayerController.ts lines 48-143 and 293-340.

Prompt:

"mario can't move even i am focused on the web page"

AI response / generated assistance: ChatGPT found that the Cocos physics manager needed to be enabled and that player movement depended on `cc.RigidBody.linearVelocity`. It added physics enabling in GameManager and adjusted controller logic so the player could move and jump.`

Refinement and explanation: I tested in the browser preview, confirmed keyboard focus, and adjusted player move speed and jump force for gameplay feel.

5. Question block and mushroom interaction

Location: BlockController.ts lines 24-169; MushroomController.ts lines 16-99; PlayerController.ts lines 237-252.

Prompt:

"implement the interaction of question mark block(when touching the questionmark block from downwards, it generates a mushroom, and when player eats the mushroom, it becomes large."

AI response / generated assistance: ChatGPT implemented question block hit detection, mushroom spawning, mushroom movement, mushroom collection, and player growth. Several follow-up fixes were generated because the mushroom initially appeared at the wrong time or did not spawn reliably.

Refinement and explanation: I connected the Mushroom prefab in the Cocos inspector and tested hitting the block from below. I also verified that the mushroom no longer pushes enemies and that it disappears after collection.

6. Enemy interaction

Location: EnemyController.ts lines 36-192; PlayerController.ts lines 143-209 and 253-292.

Prompt:

"implement interaction of enemy, decrease life when player get hits by enemy, and kill the enemy when player jumps onto its head."

AI response / generated assistance: ChatGPT implemented enemy patrol, side-hit damage, stomp detection, enemy defeated state, score gain, and reset behavior. It also later disabled physical pushing between Mario and the enemy so dead Mario would not be pushed around.

Refinement and explanation: I tested side collision and stomp collision. I reported bugs where Mario sometimes failed to respawn, and the respawn flow was revised to delay respawn and reset collider/rigidbody state.

7. Goal flag and level clear

Location: GoalFlagController.ts lines 10-63; GameManager.ts lines 273-282.

Prompt:

"touching the goalflag happens nothing"

AI response / generated assistance: ChatGPT added contact-listener setup, forced the goal flag collider to sensor mode, and added a backup bounding-box overlap check. The goal flag now calls the GameManager level clear function.

Refinement and explanation: I tested touching the goal flag in play mode and verified that the level clear panel appears.

8. UI, camera, background, and placeholder icon fixes

Location: CameraController.ts lines 33-73; GameManager.ts lines 78-118; Level1.fire scene settings.

Prompt:

"the ui goes away again" and "remove the image icon and its collider in the middle of the scene"

AI response / generated assistance: ChatGPT fixed camera/background/UI behavior and added cleanup for placeholder scene nodes that caused unwanted icons/colliders in the middle of the scene.

Refinement and explanation: I adjusted UI positions, background setup, and ground visibility in Cocos Creator. The final project keeps UI visible and removes the unwanted middle icon.

9. Sound effects

Location: GameManager.ts lines 288-305; PlayerController.ts lines 25-31 and sound calls; EnemyController.ts stomp sound; BlockController.ts hit sound; MushroomController.ts collect sound.

Prompt:

"implement animation of player and enemy, also do the sound effect part, and there is a small bug, please reset life to initial value when restarting"

AI response / generated assistance: ChatGPT generated code hooks for BGM and sound effects such as jump, damage/death, power-up, question block, enemy stomp, game over, and level clear. However, the final project does not include actual animation clips, so animations are not claimed in the README.

Refinement and explanation: I kept the sound effect functionality and explicitly documented that animations are not implemented. I also verified that restarting resets life to the initial value.

Explanation of Final Logic

Game process: GameManager stores the current game state and switches between menu, level select, playing, game over, and level clear. It also updates the score, life, and timer labels. Restart resets score/life/timer and emits reset events so objects return to their initial states.

Player: PlayerController listens for keyboard input and sets the rigid body velocity for movement. Jumping applies upward velocity when grounded. Enemy side contact causes damage, while landing on the enemy causes a stomp. Falling below the death boundary also causes life loss and respawn.

Question block: BlockController checks whether Mario hits the block from below. If the block has not been used, it spawns a mushroom above the block, plays a sound if assigned, gives score, and changes the block state so it cannot be used again until reset.

Mushroom: MushroomController moves the spawned mushroom horizontally and checks for overlap with Mario. When collected, it calls the player power-up logic, plays a sound if assigned, and removes the mushroom.

Enemy: EnemyController makes the Goomba patrol within a limited range. If Mario stomps it, the enemy is disabled and score is added. If Mario touches it from the side, Mario takes damage.

Goal flag: GoalFlagController uses both contact callbacks and bounding-box overlap as a backup to detect when Mario reaches the goal, then calls GameManager.levelClear().

Sound: GameManager handles background music and provides helper functions for sound effects. Individual scripts call their assigned sounds during jump, damage, mushroom, block, stomp, game over, and level clear events.

Student Modification Statement

AI-generated code was not submitted blindly. The code was repeatedly tested inside Cocos Creator 2.4.8, manually connected to scene nodes, and revised based on actual runtime behavior. Scene setup, sprite assignment, collider sizing, prefab connection, UI layout, button handlers, Firebase deployment preparation, and final feature documentation were adjusted by the student.

Non-Usage / Not Claimed

The final project does not claim completed player/enemy animation scoring. Bonus features such as membership, leaderboard, multiplayer, and backend server are also not claimed unless separately implemented by the student after this report.